The Journal of Systems and Software 231 (2026) 112625

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Studying SATD in drone systems with Human-AI collaboration ☆

Leevi Rantala [a,*], Lwin Khin Shar [b], Mika V. Mäntylä [c], Wei Minn [b], Yan Naing Tun [b]

[a] University of Oulu, Oulu, Finland
[b] Singapore Management University, Singapore, Singapore
[c] University of Helsinki, University of Oulu, Helsinki, Oulu, Finland

ARTICLE INFO

Dataset link: https://figshare.com/s/1870bcc7e6c1cefff63c

Keywords:
Self-Admitted Technical Debt
Technical debt
Drones
Large-language models
Sample study

ABSTRACT

Background: Self-Admitted Technical Debt (SATD) refers to sub-optimal solutions that developers acknowledge within the source code. SATD research originated on Java projects but is expanding to other domains. We focus on SATD in drones, which are used for various critical tasks.
Aims: The primary objective is to investigate SATD in drone systems. The second aim is to explore the integration of AI and human collaboration for SATD labelling and classification.
Method: Method: We conducted a sample study of SATD comments in drone systems (14 open source, 4 SDKs) to analyse the quantity and types of SATD comments present. Our study incorporates collaboration between AI and humans by utilising LLM for SATD classification. Additionally, we classified a sample of 385 SATD comments as either drone-specific or non-drone-specific.
Results: The most prevalent SATD categories in drone software are Code Debt (35%), Unclassifiable Debt (16%), and Design Debt (15%). We found that 22% of SATD is specific to drones. Drone-specific SATD is proportionally more focused on Requirements and Design Debt compared to non-drone-specific SATD. We found that using both human and LLM for SATD classification can improve accuracy, as both LLM and human revised their initial ratings. After two rounds, a "near-perfect agreement" (Fleiss' kappa 0.83) was achieved.
Conclusions: Future studies should investigate whether our observation that domain-specific (drone) SATD comments relate more to Requirement Debt holds true in other domains. We propose a workflow that integrates AI into classification tasks, enhancing the accuracy of both human and AI classifications.

1. Introduction

Technical Debt (TD) is a term used to depict sub-optimal solutions in software design and implementation with the goal of having a short-term benefit while harming software quality in the long run (Avgeriou et al., 2016). TD has been attributed to impact software maintainability and ability to evolve (Kruchten et al., 2012). TD can be classified into several different categories. Commonly used categories used include Architectural Debt, Build Debt, Code Debt, Defect Debt, Design Debt, Documentation Debt, Infrastructure Debt, Requirement Debt, and Test Debt, which were identified independently in two research works (Alves et al., 2014; Li et al., 2015). In addition to these shared categories, there exists other classification categories such as Versioning Debt, which was identified in Li et al. (2015). The work by Alves et al. (2014) adds People Debt, Process Debt, Service Debt and Test Automation Debt as categories. Finally, work done in Rios and de Mendonç Neto (2018) adds Usability Debt as an additional category.

Self-Admitted Technical Debt (SATD) comments mark code, which have "intentional quick and temporary fixes" (Potdar and Shihab, 2014). In Zampetti et al. (2021), SATD is defined as "any source code comments for annotating delayed or intended work activities". If a SATD comment is marked with a specific keyword such as "TODO" or "FIXME", we refer to it as Keyword-Labelled SATD (KL-SATD), a subclass of SATD (Rantala et al., 2020).

Since SATD was originally coined in the context of Java projects (Potdar and Shihab, 2014), much of the past research has focused on Java (e.g., Maldonado et al., 2017; Alhefdhi et al., 2024; Yu et al., 2022; Iammarino et al., 2021; Bavota and Russo, 2016). However, in the last two years, SATD studies have expanded to other languages and domains, such as Android applications (Wilder et al., 2023), Ethereum contracts (Ebrahimi et al., 2023), Docker files (Azuma et al., 2022), machine learning (OBrien et al., 2022), build systems (Xiao et al., 2024), and the R language (Sharma et al., 2022). This demonstrates the growing interest and importance of studying SATD in different contexts and domains.

☆ This article is part of a Special issue entitled: 'TechDebt' published in The Journal of Systems & Software.

* Corresponding author.
E-mail address: leevi.rantala@oulu.fi (L. Rantala).

https://doi.org/10.1016/j.jss.2025.112625
Received 30 August 2024; Received in revised form 28 February 2025; Accepted 5 September 2025
Available online 12 September 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

In this paper we focus on SATD in drone systems. Drones are crewless aerial vehicles with possibilities and capabilities for many use cases, including security and safety-critical operations such as military operations, surveillance, package delivery, and infrastructure inspection. Recently, drones have been characterised to have "a game changing effect" due to their expansive use in Russo-Ukrainian War (Kunertova, 2023; DeVore, 2023).

Accumulating TD and SATD in the software of drone systems can negatively impact the reliability of drones for carrying out operations. For example, a previous study (Kim et al., 2021) reported several input validation and control semantics bugs in drone control programs, which could lead to severe and life-threatening hazards such as the drone colliding with a train (List, 2024). Another study by Rejeb et al. (2022), lists possible challenges for full utilisation of drones, which include data security, sensor technology issues affecting the reliability or accuracy of measurements, integration complexity and substantial implementation costs. The accumulation of TD and SATD in the drone software could further impact these issues.

In addition, drone systems are embedded systems which are written predominantly in C/C++/C#. Very little research on SATD exists for projects written in C/C++/C#. The only paper examining projects written in one of the aforementioned languages focused on SATD in deep-learning frameworks, and only one project was written entirely in C++ (Liu et al., 2020). Therefore, there is a need for more studies looking SATD in C, C++, or C#.

Large-language models (LLMs) are models that have been trained with large amounts of data and different parameters (Zhao et al., 2023). LLM possess emergent abilities that can enable them to solve complex tasks which are not possible for smaller language models (Wei et al., 2022). Given that LLMs are useful in automating software engineering tasks such as code completion and bug fixing (Bird et al., 2022), this work aims to explore the feasibility of leveraging an off-the-shelf LLM tool to assist with SATD classification tasks and reduce manual effort. It aims to report how LLMs can be utilised in this context.

We seek to fill these gaps in the scientific literature by performing a sample study defined by Stol and Fitzgerald (2018). Specifically, our research seeks to answer the following research questions (*RQs*):

- **RQ1**: What is the amount of KL-SATD comments in drone systems?
- **RQ2**: Can LLM models be utilised in classifying KL-SATD comments?
- **RQ3**: What are the categories of KL-SATD comments in drone systems?
- **RQ4**: How much of KL-SATD comments are specifically related to drones?
- **RQ5**: Is there a difference in SATD categories between drone application repositories and drone SDK repositories?

We discover that approximately 1% of all the comments in the dataset are KL-SATD comments. The distribution of KL-SATD comments within projects is localised in few files, and that the growth of project size rather than number of comments in general explains better the increase in KL-SATD comments. We successfully utilise LLM models in KL-SATD classification, but special attention is needed to guarantee reliable results. The most prominent categories of KL-SATD comments are Code Debt, Unclassifiable Debt and Design Debt. Annotation of a random sample shows that approximately 21% of KL-SATD comments are related to drones, and that drone-related KL-SATD comments have statistically significant amount of more Requirement Debts in them. Finally, there is statistically significant difference between SDKs and application repositories, however this result does not hold after Bonferroni correction.

The research questions and answers help us understand KL-SATD comments in drone systems. To our knowledge, this is the first research to investigate SATD in the domain of drones, filling a significant gap in the scientific body of knowledge. Secondly, our research looks into SATD in projects written in C,C++ and C# languages, which are currently a distinct minority in SATD research.

Our paper is structured as follows: First, we go over the methodology of our study, including the building of the dataset. After that, we review the results of our questions and discuss them in relation to previous works. Finally, we discuss the threats to validity, review previous works related to ours, and offer our conclusions and future plans.

2. Research strategy

This section describes our research strategy. We first go over our study's overview and workflow and then explain each step in more detail, starting with dataset creation and ending with the annotation explanations. The RQs are answered in Section 3.

2.1. Study overview

We perform a sample study as defined by Stol and Fitzgerald (2018). The purpose is to generalise over certain *actors*, which can be software development artefacts. Sample studies have been used to study, for example, the effect of coding language on code quality (Ray et al., 2014). In our study, the actors are open-source software projects in the drone domain written in C, C++ or C#.

The workflow of our study is presented in Fig. 1, which includes four major steps:

- Step 1: Creating the dataset by parsing the repositories and pre-processing the comments
- Step 2: KL-SATD classification with LLM
- Step 3: Drone-specific SATD identification and annotation
- Step 4: Analysis of the results by answering our RQs.

The numbers (24, 21, 18) in Fig. 1 indicate how many repositories were left after a step had been performed. For full details, see our replication package.[1] We will next go over each step in more detail.

2.2. Step 1: Dataset creation

Dataset Creation has four individual steps, which are Repository selection and cloning, repository parsing, comment pre-processing and finally, KL-SATD identification.

2.2.1. Repository selection/cloning

We created the dataset by selecting and cloning the 20 most popular repositories from GitHub in terms of stars, which matched the search term "drone". Stars are used in Github to mark interesting projects, and they have been used earlier to define projects' popularity (Ray et al., 2014). We also included the four official SDK-kits published by DJI,[2] which is the largest global drone manufacturer (Statista, 2023). This was done in order to be able to contrast drone application repositories against SDK-kits. Thus, our initial dataset includes 24 repositories.

2.2.2. Repository parsing

We parsed all the repositories utilising srcML-tool (Collard et al., 2013). It translates source code written in C, C++, C# or Java into XML notation. This notation includes tags presenting abstract syntax elements of code, allowing for identifying different elements like comments. There were three repositories where srcML parsing did not work as expected. Instead of completing the parsing, the program got stuck indefinitely, did not complete, and finally had to be shut down manually. No explanation or error report was produced afterwards, so we

[1] https://figshare.com/s/1870bcc7e6c1cefff63c.
[2] https://www.dji.com/.

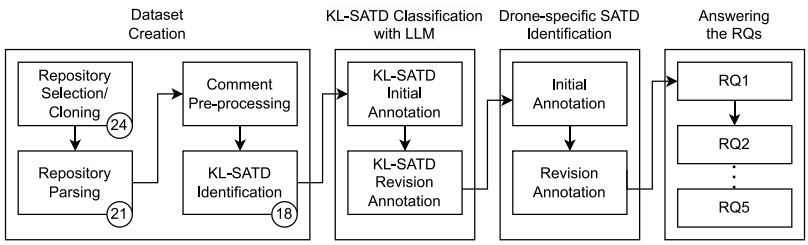

Fig. 1. Workflow of our study.

cannot explain why this happened. We tried to parse these repositories repeatedly, always with the same end result. After excluding these three repositories where srcML parsing failed, we were left with 21 repositories.

2.2.3. Comment pre-processing

Source code comment pre-processing involved two sub-steps: first, grouping related individual comments and then pre-processing of the comments. To determine whether comments should be grouped, we first identified all the comments located in lines following each other. Then, if they started from the same indentation, we concluded them to be part of the same comment. After grouping, we created a pre-processed version of all the comments by removing the non-alphanumeric characters from the messages and removing excess spaces. Both the unprocessed comments as well as the processed comments are retained in the final dataset.

2.2.4. KL-SATD comment identification

Our work focuses exclusively on Keyword-Labelled Self-Admitted Technical Debt (KL-SATD), as we want to include only true SATD comments in our study. The code comments were identified as KL-SATD comments by examining whether one or more of the following words was present in the comments: "TODO", "FIXME", "HACK", or "XXX". These words were chosen as they have been used in previous research to mark comments as SATD (Rantala et al., 2020; de Freitas Farias et al., 2015; Storey et al., 2008; Maldonado and Shihab, 2015). Three repositories did not have any of these keywords in them. As we are interested exclusively in KL-SATD, these repositories were filtered out, leaving 18 repositories in the final dataset.

2.2.5. Final dataset

In our final dataset, there are 18 projects with 16,666 files. From these, 8599 are written in C, 7946 in C++, 12 in C#, and 109 in Java. Overall, more than 99% of the files were written in C or C++. Table 1 shows the statistics of the analysed repositories. It lists the repository name, creation year, number of commits, number of files per programming language, number of SATD and non-SATD comment lines, and lines of code. The line of code information was counted from the projects' files where parsing was successful.

2.3. Annotation procedure

In order to perform the two tasks of classifying the KL-SATD comments into Technical Debt categories with LLM, and for identifying Drone-specific SATD comments, we used a similar annotation processes for both of these tasks. A systematic annotation process was used in order to guarantee the reliability of the results (Carletta, 1996). It had the following workflow:

1. Individual annotation
2. Fleiss' kappa calculation
3. Disagreement Discussion
4. Agreement adjustments for the final dataset

We will next go over each step in order, and discuss the possible specifics related to the two tasks.

2.3.1. Individual annotation

In the first step, each annotator annotated the dataset given to them individually without discussing any results with the second author. For KL-SATD annotation, authors were tasked to label the all the KL-SATD comments to a SATD category (see Section 2.5.1). In this case, ChatGPT 3.5 took over the annotation duties of the second annotator. More details from this step are explained in Section 2.5.

For Drone-specific SATD identification, the annotators were tasked to label a sample, deciding whether SATD comment is specifically related to drones. More information on sample creation can be found on Section 2.4. If the SATD comment could be generalised to a more generic software issue, then it was deemed not to be specifically related to drone.

2.3.2. Fleiss' kappa calculation

After this initial annotation round, we calculated the author agreement using Fleiss' kappa. Fleiss' kappa considers the possibility that agreements in the rating procedure can happen by chance, and it is suitable for classifications done on a nominal scale. Although there are no strict guidelines for interpreting Fleiss' kappa, we use the scale set in earlier work by Landis and Koch (1977) as a guideline. The kappa statistic and strength of agreement are presented in Table 2.

Generally, these benchmark numbers can be used for annotation regardless of the number of categories used.

2.3.3. Disagreement discussion

For the third step, a discussion was held between the annotators in order to resolve any disagreements between them. For KL-SATD annotation, ChatGPT 3.5 was given a revised discussion prompt, while the human annotator revised their annotations based on the answers given by ChatGPT 3.5 in the initial annotation. Again, more details from this step are explained in Section 2.5. As full agreement could not be reached at this point, the final dataset includes both the human and ChatGPT 3.5 annotations.

2.3.4. Agreement adjustments for the final dataset

In the fourth step, in order to resolve the disagreements between the human annotator and ChatGPT, a second human annotator performed an independent annotation round for these disagreed cases. Afterwards, the human annotators held a final discussion round over these revised annotations in order to for the final dataset.

For Drone-specific annotations, the two human annotators sat down with each other to solve their disagreements and to agree on a consensus. The final dataset includes their agreed labels of what constitutes as a drone-specific KL-SATD comment.

2.4. Drone-related KL-SATD comment identification

In order to determine how many KL-SATD comments relate directly to drones, we decided to annotate a sample from all of the comments. The following section describes the process to determine the sample size. To guarantee a confidence interval of 95% we determined the sample size using the following formula:

$$n = p(1-p)(\frac{Z}{E})^2 \quad (1)$$

Table 1
Final dataset summary.

Repository name	Year created	Commits	File language				Number of comments		LOC
			C	C++	C#	Java	non-SATD	SATD	
ardrone_autonomy	2012	296	8	4	0	0	132	10	5070
ardupilot	2010	58,215	1420	1375	0	0	78,010	286	702,553
clover	2017	3260	2	28	0	0	851	35	46,234
dRehmFlight	2020	93	20	10	0	0	2558	62	21,855
DroneBridge	2017	371	390	1	0	0	9037	13	134,121
ExpressLRS	2018	1604	181	113	0	0	3628	27	47,507
Faster	2016	574	24	44	0	0	1314	7	13,752
GAAS	2019	765	2657	3355	0	0	100,950	1710	1484,756
mav_active_3d_planning	2018	267	79	77	0	0	1581	9	13,395
openMVG	2013	2293	1054	861	0	0	52,821	722	742,284
polygon_coverage_planning	2018	524	37	43	0	0	834	9	10,832
Prometheus	2019	1323	76	707	0	0	9627	86	139,994
PX4-Autopilot	2011	42,400	1505	1942	0	0	54,001	798	799,929
qgroundcontrol	2010	18,500	576	414	0	19	15,371	160	260,192
Mobile-SDK-Android	2015	185	0	0	0	113	531	4	23,727
Onboard-SDK	2015	1346	247	298	0	0	16,637	85	180,431
Payload-SDK	2019	81	356	21	0	0	28,670	2	193,045
Windows-SDK	2018	23	376	18	14	0	8629	58	100,536
Total:		132,097	9558	9317	18	132	385,182	4083	4,920,213

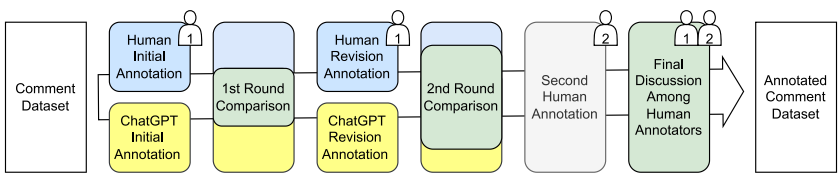

Fig. 2. KL-SATD comment classification workflow.

Table 2
Kappa and strength of agreement (Landis and Koch, 1977).

Kappa	Strength of agreement
<0.00	Poor
0.00–0.20	Slight
0.21–0.40	Fair
0.41–0.60	Moderate
0.61–0.80	Substantial
0.81–1.00	Almost perfect

Here, p is the unknown population proportion, Z is the value to be used from the standard normal distribution reflecting the desired confidence interval, and E is the desired margin of error. We used 0.5 for the value of p, which maximises the sample size. This gives us a sample size of 385 cases with a 95% confidence interval.

We utilise Fleiss' kappa introduced in Section 2.3 to calculate the agreement between the raters. The Fleiss' kappa after the initial independent round with the two annotators was 0.51 (moderate agreement). If the kappa had not shown agreement, we would have deduced that there does not exist drone-related KL-SATD that can be independently identified. However, this initial result indicates that there exists drone-specific SATD comments. Afterwards, the authors met to discuss their disagreements and reach a consensus on the results.

2.5. KL-SATD classification with LLM

KL-SATD Comment Classification to categories was done independently by two annotators. However, in this case, ChatGPT 3.5 took on the duties of the second annotator. The process is shown in Fig. 2. We will next go over each step in the process.

For the initial annotation, the human annotator was given the list of possible categories of technical debt and their descriptions presented in Section 2.5.1. ChatGPT was given the same descriptions in a prompt and it was tasked based on these descriptions to label each SATD comment into one of these categories or as 'No Debt' comment, if it was unable to give it any other label. It was also tasked to give an explanation as to why that comment belongs to that category.

After the initial annotation round, we calculated Fleiss' kappa statistic to see whether it was feasible to continue to the disagreement discussion. Too low score would mean that ChatGPT was unable to classify comments in a meaningful way. The Kappa values for the whole classification process are presented in more detail in Section 3.2.

The disagreement discussion was done with a revision annotation round. This time ChatGPT was given the category classifications given by the human annotator and ChatGPT. It was tasked to explain why each of these categories is a valid option, and then classify the SATD comment to one of these categories. Meanwhile, the human annotator looked into the explanations ChatGPT had given to its comments in the initial annotation round and possibly revised their answers based on these. This process simulates the discussion between humans, where each party present the arguments for her/his choices and one considers whether or not the arguments by another party have valid grounds. Examples of classified comments and ChatGPT explanations are given in Section 3.2.

After the first and second rounds, we calculated the rater agreement between the human annotator and ChatGPT utilising Fleiss' kappa. There might still be disagreements left, where neither the human nor ChatGPT would have changed their answers. No further rounds were held between the human annotator and ChatGPT, as the revision prompt would have stayed the same.

As there were still some cases where the human annotator and ChatGPT disagreed, we needed to resolve these in an unbiased manner. Second human performed an extra annotation round independently on these cases. This second annotator is one of the other authors of this study, who did not participate in the original annotation round. We call this person the second human annotator. He was given the conflicting 574 labels which were anonymised and randomly mixed in order to reduce the bias to choose either annotators choice by default.

After this, the original human annotator and the second annotator had a final discussion round on the disagreements to reach a consensus. This results in the final ground truth on the classification. In order to provide transparency, we also include the answers given by ChatGPT in the replication package.

2.5.1. KL-SATD classification categories

We classify the KL-SATD comments into different technical debt categories, which were commonly identified in two independent research works (Alves et al., 2014; Li et al., 2015). We exclude Infrastructure Debt from our study as it relates to infrastructure issues in the organisation, not issues in the software itself. On the other hand, we include an additional category called Unclassifiable Debt (Wilder et al., 2023; Azuma et al., 2022). Next, we present the classifications used in our study along with their explanations.

Architectural Debt refers to "problems encountered in project architecture, for example, violation of modularity, which can affect architectural requirements such as performance, robustness, etc". Li et al. (2015), or "architecture decisions that make compromises in some internal quality aspects such as maintainability" (Alves et al., 2014).

Build debt refers to "build related issues that make this task harder, and more time/processing consuming unnecessarily" including unnecessary code and ill-defined dependencies making the process slower than it should be (Li et al., 2015), or "flaws in a software system, in its build system, or in its build process, which make the build overly complex and difficult" (Alves et al., 2014).

Code Debt refers to "problems found in the source code which can affect negatively the legibility of the code making it more difficult to be maintained" including issues related to bad coding practices (Li et al., 2015), or "poorly written code that violates best coding practices or coding rules. Examples include code duplication and overly complex code" (Alves et al., 2014).

Defect Debt refers to "defects, bugs, or failures found in software systems" (Li et al., 2015) or "known defects [...] but due to competing priorities, and limited resources have to be deferred to a later time" (Alves et al., 2014).

Design Debt refers to "technical shortcuts that are taken in detailed design" (Li et al., 2015) or "use of practices which violated the principles of good object-oriented design" (Alves et al., 2014).

Documentation Debt is "insufficient, incomplete, or outdated documentation" (Li et al., 2015) or "missing, inadequate, or incomplete documentation" (Alves et al., 2014).

Requirement Debt is the "distance between the optimal requirements specification and the actual system implementation" (Li et al., 2015) or "tradeoffs made with respect to what requirements the development team need to implement or how to implement them" (Alves et al., 2014).

Test Debt refers to "shortcuts taken in testing" (Li et al., 2015) or to "issues found in testing activities which can affect the quality of testing activities" (Alves et al., 2014).

Unclassifiable Debt refers to comments which are "too vague" to be classified into any category (Azuma et al., 2022). They might also be "incomplete "or "not descriptive enough" (Wilder et al., 2023).

3. Results

In this section, we present the results of our five research questions in order.

3.1. RQ1. What is the amount of KL-SATD comments in drone systems?

We answer our first research question in three parts. First, we take a look of how the KL-SATD comments are distributed in the projects. Secondly, we investigate whether there are statistical differences between projects when looking at KL-SATD distribution in them. And finally, we investigate whether the project size or the number of non-SATD comments can be used to predict KL-SATD comment numbers.

Table 3
Summary of SATD comments in the dataset.

	SATD Comments			
	Min.	Median	Mean	Max.
Per Project	2	46.5	226.83	1710
Per File	1	1	2.045	25
Files w. Comments	1	12.50	110.94	755

Table 4
Summary of non-SATD Comments in the Dataset.

	Non-SATD Comments			
	Min.	Median	Mean	Max.
Per Project	132	8833	21,399	100,950
Per File	1	9	23.12	7172
Files w. Comments	10	359	814.94	3845

3.1.1. KL-SATD distribution in projects and files

Our dataset includes 18 projects in total. These projects contain 389,265 comments. From these comments, 385,182 are comments without any KL-SATD features, while 4083 comments (1%) have a KL-SATD feature. The comments range from presenting problems in the code ("`TODO duplicated from above`") to defects ("`FIXME: Temporary workaround for a bug in slicing of bool tensors`") and architectural problems ("`TODO: what is this modules purpose?`"). Full list of SATD categories and more examples are presented later in Section 3.3.

The distribution of KL-SATD comments between projects varies greatly. The median comment count for SATD comments per project was 46.5, and 8833 for non-SATD comments. Meanwhile, the mean comment count for SATD comments per project was 226.83; for non-SATD comments, it was 21,399. The results in both cases mean that there were few big projects, which skewed the mean result.

If we look within the projects, we can count how many files had KL-SATD and non-SATD comments. The median amount of files with KL-SATD comments within projects was 12.5, while the mean amount was 110.94. Similarly, for non-SATD comments, the median amount of files with non-SATD comments was 359 per project, and the mean amount was 814.94. The result indicates that KL-SATD comments were not evenly distributed among the files within projects, instead they were concentrated in fewer files.

Lastly, we looked at how many KL-SATD comments the files contained. From all the files with KL-SATD comments, the median amount of KL-SATD comments per file was 1, and the mean amount was 2.045. Similarly, for non-SATD comments, the median amount of comments per file was 9 comments, while the mean was 23.12. The results show that, on average, there were only around 2 KL-SATD comments per file. The information is summarised in Tables 3 and 4.

Finding 1: KL-SATD comments constitute approximately 1% of all comments in the dataset. The KL-SATD comments are localised in a few files within projects. Furthermore, these files generally have only 1-2 KL-SATD comments.

3.1.2. Are there statistical differences between projects in KL-SATD distributions?

As the KL-SATD comments are located in few files within the projects, we wanted to examine whether there are statistical differences between projects and the files and KL-SATD comment amounts. We do this by summing up the KL-SATD comment amounts in files and then compare these distributions using Kruskall-Wallis test with the null hypothesis that there are no differences in distributions between projects. The results show a statistical difference in the ratio of KL-SATD numbers per file between the projects $\chi^2(17) = 55.307, p = 6.116e-06$. Therefore, we reject the null hypothesis and conclude that

Table 5
Project pairs with statistically different KL-SATD distributions.

Project pair	p-value
PX4-Autopilot & GAAS	0.00011
PX4-Autopilot & OpenMVG	0.01498
Prometheus & GAAS	0.02004

there are differences between projects when looking at the files and KL-SATD comment amounts in them.

However, we do not know yet that between which projects do the differences exist. To investigate further, we perform pairwise comparisons using Wilcoxon rank sum test with sequential Bonferroni correction, which has increased statistical power over standard Bonferroni correction (Rice, 1989). In our case, there are three pairs which might be under statistical significance level ($p < 0.05$). These are presented in Table 5, along with their p-values.

However, in order to pass sequential Bonferroni correction, the p-value of the lowest pair must be under $\frac{p}{n}$, second lowest must be under $\frac{p}{n-1}$ and the third under $\frac{p}{n-2}$, where n is the number of pairwise comparisons. Since we have 18 projects, the number of pairwise comparisons n equals $\frac{m(m-1)}{2}$, where m is the number of projects. In total, we have 153 comparisons ($n = 153$). The only pair which passes sequential Bonferroni correction is PX4-Autopilot & GAAS. Therefore, we can conclude that only these two projects differ from each other when looking at the number of KL-SATD comments per file.

Table 6 sums up the KL-SATD comment distribution in these two projects. The columns show how many files have that many comments in them. For example, GAAS has 408 files and each of the file has 1 KL-SATD comment, PX4-Autopilot has 30 files and each of the file has 3 KL-SATD comments, etc.

When comparing the distributions, we can see that both projects have the same minimum (1) and median (1) number of KL-SATD comments per file as the whole dataset presented in Table 3. However, the mean for GAAS is slightly higher (2.265) while the mean for PX4-Autopilot is slightly lower (1.739), when compared to the whole dataset. The highest number of KL-SATD comments in one file for GAAS is 25 comments, while for PX4-Autopilot the maximum is 16 KL-SATD comments. In total, GAAS has a longer tail in the distribution towards the high end of KL-SATD comment amounts, meaning there are more files with much higher KL-SATD amounts than in PX4-Autopilot.

Finding 2: There exists statistically significant differences between projects when looking at KL-SATD comment distributions in files. However, statistical difference in the distribution can only be shown between PX4-Autopilot and GAAS.

3.1.3. Can project size or non-SATD comments predict KL-SATD comment amounts?

We want to investigate whether there is any correlation between the KL-SATD comments and the project size or the amount of non-SATD comments. We chose LOC as our size measure, as it has been shown in the past to be an important feature regarding SATD (Zampetti et al., 2017).

We ran Spearmann's correlation and discovered that KL-SATD had a statistically significant correlation for both LOC ($\rho = 0.74, p < 0.001$) and the number of non-SATD comments ($\rho = 0.72, p < 0.001$). The results indicate that the number of SATD comments is correlated to both the project size and the number of non-SATD comments.

To further investigate whether LOC or the number of non-SATD comments is the better predictor for SATD comment amount, we ran several linear regression models. We used both LOC and the number of non-SATD comments as predictors. We discovered that LOC and non-SATD comments have a variance-inflation factor (VIF) of 15, suggesting a severe multicollinearity problem. This was confirmed with a regression model including both LOC and number of non-SATD comments as predictors, as the coefficient for number of non-SATD comments was negative, suggesting overfitting. The model is presented in Table 7.

Due to the high VIF, we ran linear regressions utilising only LOC or non-SATD as a predictor for SATD comments. The regression results are presented in Tables 8 and 9.

In both cases, the coefficients were positive. This means that an increase in LOC or number of non-SATD comments correlates with the increase in SATD comments. The linear regression results show that it is better to use LOC rather than the number of non-SATD comments to predict SATD comment amounts, as evident from their R^2 scores (0.9222 vs. 0.7496). Including both of these to the model does increase the R^2 score to 0.9824, but in this case the model is suffering from overfitting as the number of non-SATD comments has a negative coefficient.

Finding 3: Linear regression results show that LOC is the better predictor for SATD comment amounts ($R^2 = 0.9222$) than the number of non-SATD comments ($R^2 = 0.7496$). This means that growth in project size, rather than in the number of non-SATD comments, can explain better about the increase in SATD comment numbers.

3.2. RQ2: Can LLM models be utilised in classifying KL-SATD comments?

Following the procedure described in Section 2.5.1, one of the authors and ChatGPT 3.5 proceeded with the initial annotation round. After this round, the inter-rater agreement between human annotator and ChatGPT had Fleiss' kappa of 0.388 ($p < 0.001$), indicating fair agreement between the raters. The result indicates, that it is feasible to continue, as it demonstrates fair agreement between ChatGPT 3.5 and the human annotator.

After the initial round, the author and ChatGPT had a disagreement discussion round to resolve the discrepancies in the classifications between the author and ChatGPT. The human annotator changed his annotation category for 44 (1%) comments from 4083 comments, agreeing with ChatGPT 3.5 rather than his initial category. ChatGPT 3.5 chose a different category for 1481 (36%) comments, choosing a category given by the human annotator rather than what was chosen in the first round.

The Fleiss' kappa after both ChatGPT and the human revised their answers was 0.83 ($p < 0.001$), indicating near-perfect agreement. The result shows significant agreement between the raters.

The final disagreement cases were handled by second human annotator. The initial agreement between the second human annotator and original human annotator was 0.572, and between second human annotator and ChatGPT the kappa was 0.07. The results indicate that second human annotator preferred the answers given by the original human annotator over those given by ChatGPT. Finally, the human annotators had a discussion round to resolve all the final disagreements and formed the final dataset.

Finding 4: Utilising LLM to assist in SATD comment classification is possible. However, to guarantee reliable classification results, a disagreement discussion classification round is advisable.

3.2.1. Qualitative examples of resolved disagreements with LLM

There were cases where the human rater had either understood the comment differently than ChatGPT 3.5 or cases where there had been an oversight by the human rater. An example from the former is a comment "`TODO: handle auto missions without NAV_TAKEOFF mission cmd`", which the human had thought referring to a design which uses a command that is not necessary to be used. However, ChatGPT 3.5 had labelled it differently, stating that "*It is difficult to definitively determine the best-fit category without further context or information. However, based on the provided comment, the most fitting category would be Requirement Debt, as it directly refers to a specific requirement that needs to be addressed.*" After reading this explanation, the human rater agreed with this explanation. The comment concerns a larger implementation need, meaning we must handle these missions. It

Table 6
KL-comment distribution for GAAS and PX4-Autopilot.

Project	Number of files with # SATD Comments																
	1	2	3	4	5	6	7	8	9	10	12	13	14	16	19	21	25
GAAS	408	171	55	46	21	18	10	7	2	2	4	5	1	0	1	3	1
PX4-Autopilot	313	77	30	10	9	8	4	4	0	2	1	0	0	1	0	0	0

Table 7
Predicting SATD with linear regression using LOC and non-SATD.

Coefficient	Estimate	Std. Error	t	p-value
(Intercept)	−3.845e+01	1.815e+01	−2.119	0.0512
LOC	2.071e−03	1.469e−04	14.093	4.67e−10
Non-SATD	−1.406e−02	1.962e−03	−7.166	3.25e−06
R^2 = 0.9824				

Table 8
Predicting SATD with linear regression using LOC.

Coefficient	Estimate	Std. Error	t	p-value
(Intercept)	−6.096e+01	3.640e+01	−1.675	0.113
LOC	1.053e−03	7.645e−05	13.772	2.73e−10
R^2 = 0.9222				

Table 9
Predicting SATD with linear regression using non-SATD.

Coefficient	Estimate	Std. Error	t	p-value
(Intercept)	−44.307029	66.288524	−0.668	0.513
Non-SATD	0.012671	0.001831	6.920	3.45e−06
R^2 = 0.7496				

is not simply a comment about poor implementation. It is worth noting that ChatGPT 3.5 also expressed uncertainty but was bent on choosing Requirement Debt as the classification.

The second case, where the human changed the classification, can be described as an unintentional oversight. One such case is comment "`position limit - TODO apply circular limit`", was originally labelled as Design and Requirement Debt by the human annotator. However, ChatGPT labelled it as Code Debt, to which the human rater agreed. There is no explicit requirement here, albeit the phrase "`apply circular limit`" could be understood as one. However, the comment points more to a minor issue than an actual unimplemented requirement or a flaw in the design, therefore the human rater changed their annotation to match the one given by ChatGPT.

Meanwhile, there were several cases where ChatGPT had changed its classification to match the one given by the human annotator. One example of such comment is "`// FIXME: Perform current integration if the remaining capacity cannot be requested`", which the human had labelled as a Defect Debt and ChatGPT as Requirement Debt. In the second round, ChatGPT changed its rating to Defect Debt, giving as an answer: "*The given comment contains the keyword FIXME, which indicates a known defect or issue in the code that needs to be fixed. The comment suggests that there is a problem with the current integration in a scenario where the remaining capacity cannot be requested. This defect needs to be addressed to ensure the integration works correctly. Therefore, the comment can be classified as Defect Debt, as it highlights a specific defect that needs to be resolved*".

Another example of such a case is the comment "`// FIXME: do the whole subdev dance here and get an actual FD to do an ioctl on?`" Human annotator had given this comment a label of Design Debt, whereas ChatGPT had labelled it as Unknown Debt. When prompted again to choose between these two categories, it chose the one given by the human with the following explanation: "*Based on the given comment // FIXME: do the whole subdev dance here and get an actual FD to do an ioctl on?, it seems to contain the keyword "FIXME" which indicates that there is a known issue or a problem that needs to be fixed. Looking at the content of the comment, it appears to be related to the implementation of a specific function or feature, suggesting a design or code problem. The mention of "do the whole subdev dance here" implies that there might be a complex or convoluted design or implementation in the code. Considering these factors, the most suitable classification for this comment would be Design Debt. This is because it indicates that there might be technical shortcuts taken in the detailed design or the use of practices that violate good design principles*".

3.2.2. Qualitative examples of unresolved disagreements with LLM

As evidenced by the Fleiss' kappa, there are still some cases where the human and ChatGPT disagree. One example of such a case is the comment "`FIXME: Reference frame is not corrected for here`", which the human labelled as Code and Defect, while ChatGPT labelled as Unknown. The human annotator remains adamant that 'not corrected' can lead to defects or cause other issues later, like unnecessary code to fix the issue. Another example is "`TODO: make these parameters:`", where the human annotator labelled the comment as Code/Design Debt, while ChatGPT labelled it as Unclassifiable. In both of these cases, the second human annotator agreed with the labels given by human annotator.

We can see where the differences lie by cross-tabulating the categories between the human annotator and ChatGPT. The results are presented in Table 10.

Examining the results from Table 10, we see a strong agreement between the human annotator and ChatGPT. The largest single category which stands out in disagreement is "Unclassifiable" from ChatGPT. In several cases, ChatGPT labelled it as "Unclassifiable" instead of labelling the comment into a category. The largest misclassification category was Code Debt, where ChatGPT labelled 242 cases as Unclassifiable Debt.

Finding 5: Agreement between human annotator and ChatGPT 3.5 varies between SATD categories. ChatGPT 3.5 is prone to classify SATD as "Unclassifiable", especially when classifying comments classified as Code or Design Debt.

3.3. RQ3. What are the categories of KL-SATD comments in drone systems?

To answer our question, we counted the occurrences of different categories obtained from the labelling process for RQ2. Similarly to previous work by OBrien et al. (2022), if a comment could be classified into several categories like Code and Design debt, we count the comment as an occurrence in both categories. Next, we will review our findings from the categories in descending order of occurrences.

Code Debt (n = 1516). The most numerous category in our dataset is Code Debt, and it constitutes 33% of all SATD category counts. Code Debt is a broad category comprising several different types of problems related to problems in the code. Some examples of these comments are explicit declarations of poorly written code ("`this is a hack`"), reminders to do something in the future ("`TODO: ignore for now. I will be UDP in future.`"), notification of code duplication ("`TODO duplicated from above`"), notes related to optimisation ("`TODO optimise it for SSE3 and 4`"), and notes about unused code ("`TODO those are unused now`").

Table 10
Cross-tabulated final SATD categories.

	ChatGPT										
		Arc.	Build	Code	Defect	Design	Doc.	No Debt	Req.	Test	Unc.
Human Annotator	Architectural	75	7	11	0	4	1	0	7	0	7
	Build	0	65	6	2	1	0	0	1	1	2
	Code	2	5	1258	12	61	1	6	22	1	148
	Defect	0	1	15	464	7	2	0	12	0	43
	Design	2	4	54	12	449	1	1	57	0	120
	Documentation	0	0	1	1	0	55	0	1	0	0
	Requirement	4	1	10	10	50	5	1	473	0	63
	Test	0	1	4	2	0	1	1	4	97	7
	Unclassifiable	0	2	5	6	6	1	0	3	0	864

Unclassifiable Debt (n = 887). The second largest category is Unclassifiable Debt, making up 19% of all SATD types. This category includes all the SATD comments, which are too vague to be classified into any category. Some examples of such comments are "`TODO: home?`", "`FIXME check the Has*`", "`zx-todo`", and "`TODO: fill in this`".

Design Debt (n = 700). The third category is Design Debt, which covers 15% of all SATD types. These comments expose issues related to design on a larger scale than the Code Debt issues. Examples include relocating functionality to a better location ("`FIXME: do the whole subdev dance here and get an actual FD to do an ioctl on?`"), implementation problems on function level or higher ("`Set range finder measurement noise variance. TODO make this a function of range and tilt to allow for sensor, alignment and AHRS errors`"), and larger optimisation issues ("`FIXME if StorageOrder == RowMajor this operation is not very efficient`").

Requirement Debt (n = 617). The fourth largest SATD category is Requirement Debt. It has a 13% share from all SATD types, nearly as much as Defect Debt. These comments point out several problems related to discrepancies between requirements and implementations, such as missing requirements ("`TODO: Add possibility change bitrate`"), partially implemented requirements ("`TODO: accommodate different arrangements of sent data (ONLY default supported now)`"), and implementations not following requirements ("`TODO: Normalise output data`").

Defect Debt (n = 544). The fifth largest category is Defect Debt, and it entails 12% of all SATD types. As the name suggests, these are SATD comments about defects or bugs. Some examples of comments and the issues they deal with are mentioning possible bugs caused by deviating from requirements ("`todo: this should be thrust vector heading, not yaw.`"), problems in error case handling ("`TODO: Handle properly, maybe generate a traceback`"), comments about workarounds for bugs ("`FIXME: Temporary workaround for bug in slicing of bool tensors`"), and generic comments about defects or bugs ("`TODO: thrust bug: pointer to last instead of first`").

Test Debt (n = 117). These SATD types are related to testing. They account for less than 3% of all SATD comments. These include cases such as increasing test coverage ("`TODO: Add sampling tests`"), noting the lack of testing ("`TODO: this is not tested`"), improving existing tests ("`TODO: Make this test work well, and use range saturation function. [...]`"), and workarounds and hacks related to testing ("`Nasty hack to expose controller to unit test code`").

Architectural Debt (n = 112). These comments account for less than 3% of SATD types. The comments refer to problems larger in scope than those in Design Debt. The low number of occurrences might indicate that few architectural issues exist in the projects. On the other hand, being larger in scale than Code or Design Debts, it would be expected that the number of Architectural Debt is considerably lower. However, when there are architectural issues, the cases may be much more severe and require more effort to be solved than Code or Design Debt. Examples include relocating code to another class or file ("`TODO: move to Util or other upper class to be used by others`"), pointing out possibly unused pieces of code ("`TODO: what is this modules purpose?`"), noting unnecessary dependencies on other modules ("`Drop the dependence on TripletSparseMatrix.`"), and improving the architecture of the project ("`TODO make modular`").

Build Debt (n = 78). This category refers to problems manifesting during the build process. Examples include workarounds or hacks to ensure the build succeeds ("`TODO Apparently cannot remove this or the build fails.`"), workarounds or hacks for specific build-related issues ("`FIXME: This is only required if the library was built using SEH`").

Documentation Debt (n = 58). These are comments relating to documentation problems. The low number of occurrences compared to other SATD categories indicates that there are relatively few cases where documentation was deemed inadequate and was done adequately for most parts. However, there are cases where this can still be a significant issue. One comment refers to missing documentation when more information is needed, highlighting that this documentation has not yet been written. Examples include highlighting missing documentation ("`TODO: add a usage example`"), pointing out outdated documentation ("`FIXME sunstudio is not friendly with the above friend... META-FIXME there is no 'friend' keyword around here. Is this obsolete?`").

Finding 6: The largest SATD categories in our dataset are Code Debt (33%), followed by Unclassifiable Debt (19%) and Defect Debt (12%), totalling 64% of all classifications. The findings imply that developers create workarounds or hacks for local problems in the code, lots of comments are written in a way where understanding them without further information is impossible, and there are many possible bugs or a workaround for them.

3.4. RQ4. How much of KL-SATD comments are specifically related to drones?

The annotation results show that from our random sample of 385 comments, 82 (21.3%) are related to drones with 95% confidence interval (0.172, 0.254).

We investigated further to discover differences between SATD categories on how many comments relate specifically to drones. Table 11 lists all the SATD categories present in the sample and how many were drone-related. The final sum for different categories differs in total from the sample size, as one comment can have multiple categories assigned to it as mentioned in Section 3.3.

In total, 85 classifications (22%) were related to drones. As can be seen, there were large differences between the number of samples per category. To investigate whether there exists a statistical difference between drone-related and non-drone projects across all the categories,

Table 11
SATD categories and their relation to drone-specific SATD.

Category	Non-Drone	Drone-Related	p-value
Architectural	7 (2.3%)	1 (1.1%)	1
Build	5 (1.6%)	1 (1.1%)	1
Code	115 (37.7%)	22 (25.9%)	0.05358
Defect	46 (15.1%)	14 (16.5%)	0.736
Design	29 (9.5%)	14 (16.5%)	0.07896
Documentation	10 (3.3%)	0 (0%)	0.1268
Requirement	32 (10.5%)	21 (24.7%)	0.001915*
Test	8 (2.6%)	3 (3.5%)	0.7109
Unclassifiable	53 (17.4%)	9 (10.6%)	0.1785
Sum	305	85	<0.05

Table 12
SATD categories and their relation to applications and SDKs.

Category	Application	SDK	p-value
Architectural	113 (2.5%)	9 (5.1%)	0.04727
Build	73 (1.6%)	2 (1.1%)	1
Code	1588 (35.1%)	55 (31.3%)	0.3337
Defect	543 (12%)	15 (8.5%)	0.1908
Design	677 (15%)	26 (14.8%)	1
Documentation	52 (1.2%)	6 (3.4%)	0.02047
Requirement	631 (14%)	21 (11.9%)	0.5056
Test	120 (2.7%)	0 (0%)	0.0235
Unclassifiable	724 (16%)	42 (23.9%)	0.00893
Sum	4521	176	<0.001

we performed the Fisher's Exact Test with Monte Carlo simulation (2000 replicates). This test is suitable as there are categories with fewer than 5 occurrences, which rules out the Chi-square test. We ran the test several times to account for variability in the simulation. Each time, the result was statistically significant ($p < 0.05$). Therefore, we can conclude that there are differences between categories in how many drone-related comments there were.

Next, we performed the Fisher's Exact Test for each category separately. As we deal with multiple categories, we utilise sequential Bonferroni correction again. In our case, Requirement Debt has the smallest P value. Therefore, it must meet the criteria $P \leq \frac{0.05}{n}$ where n is the number of categories (9 in our case). Requirement Debt satisfies this criteria.

Finding 7: KL-SATD comments related to drones have a statistically significant difference of having more Requirement Debt categories in them.

3.5. RQ5: Is there a difference in SATD categories between drone applications and drone SDK repositories?

Our dataset includes both drone application and drone SDK repositories. Specifically, we included four tool repositories from drone developer DJI in our dataset. Table 12 lists the SATD categories found in application repositories and SDK repositories. Similarly to Section 3.4, the sum presents the total number of categories present and not the total number of comments. We investigated whether there are differences between drone applications and SDK repositories regarding KL-SATD amount per category. This was accomplished by dividing the projects into either applications or SDKs and calculating the KL-SATD comments per category in each group. We performed the Fisher's Exact Test between the groups with Monte Carlo simulation (2000 replicates). We again ran the test several times, and each time, the result was statistically significant ($p < 0.001$). Therefore, we can conclude that there are differences in KL-SATD categories between applications and SDKs.

To further investigate the matter, we again perform the sequential Bonferroni correction to each category with $p < 0.05$. None of the categories remain significant after the correction.

Finding 8: Overall, there is a statistically significant difference between SDK repositories and applications. However, none of the categories remained statistically significant after sequential Bonferroni correction.

4. Discussion

As our research is focused on a specific domain of drones, it might differ from other domains when looking at SATD amounts and distribution. We will next discuss our results in relation to previous research, which all investigated SATD in different domains and generic Java projects. The domains are generic Java projects (Maldonado et al., 2017), Android applications (Wilder et al., 2023), deep-learning applications (Liu et al., 2020), Ethereum contracts (Ebrahimi et al., 2023) and Docker files (Azuma et al., 2022). We will compare the amounts of SATD comments, and after that, we will look at whether the SATD categories and their proportions differ from domain to domain. Next, we will discuss utilising LLM in a classification task. Finally, we end the discussion with the implications of our work to both practitioners and researchers.

4.1. Amount of SATD comments in projects (RQ1)

The results show that from our dataset of 389,265 comments, 4083 (1%) were labelled as containing SATD. Table 13 summarises our dataset and the ones presented in previous research. It is worth noting that not all the previous papers listed the same information. Therefore, some information is missing from Table 13.

One of the earliest research papers looked into SATD in 10 different Java projects (Maldonado et al., 2017). They discovered that, on average, the projects contained 1.57% of SATD comments of all the comments in them if all of the comments were taken into account (Maldonado et al., 2017). The median number of SATD comments per project in their dataset was 271, and the mean was 407 (see Table 13). Our average number of comments is slightly lower than theirs. However, this might be due to differences between the analysed projects and the fact that we only included SATD comments containing common keywords in our dataset. This previous research looked into mature Java projects with long development histories. Our dataset is built by looking at the most popular GitHub repositories of drones, some of which have short development histories.

An exploratory study of SATD comments in Android apps shows that the median number of comments for Android applications was 4 SATD comments, while the mean was 28.92 SATD comments (Wilder et al., 2023). From all the comments in their data, 5.97% were classified as SATD comments. Compared to our dataset, their median and average comment numbers are low. However, in total, our dataset has a lower SATD percentage.

The study on deep-learning networks show that the median number of SATD comments for deep-learning framework projects was 474 SATD comments, and the mean was 1022.71 SATD comments (Liu et al., 2020). From all the comments in their 7 projects, 2.93% were labelled as SATD comments. Compared to our dataset, their numbers are overall higher than in our study.

Two studies did not report enough information to be comparable. A study into Ethereum contracts reported that the median number of SATD items in SATD contracts was 1 (Ebrahimi et al., 2023). However, they have removed cloned SATD comments from their work, so the results are not comparable to those reported here. Also, they do not report minimum or maximum numbers per contract. A recent study into SATD in Docker files (Azuma et al., 2022) found 50 SATD comments in the docker files when removing duplicates. However, they do not report any statistics on the amounts per docker file.

In summary, the results in SATD amounts in our dataset are between the ones obtained from Java projects in Maldonado et al. (2017) and those from Android applications in Wilder et al. (2023). The minimum,

Table 13
SATD comment amounts in research.

Domain	Min	Med.	Mean	Max.	Per cent
Drone projects[a]	2.0	46.5	226.83	1710	1%
Android apps[b]	1	4	28.92	17,483	5.97%
Java projects[c]	104	271	407.1	1413	1.57%
Deep-Learning[d]	54	474	1022.71	3775	2.93%

[a] Our Dataset.
[b] Wilder et al. (2023).
[c] Maldonado et al. (2017).
[d] Liu et al. (2020).

median, and mean SATD amounts in our dataset are greater than what was discovered in the Android applications but less than what was reported in Java projects. As for the maximum, we sit again between Android applications and Java projects, being much closer to the Java applications.

4.2. Utilising LLM in classification tasks (RQ2)

Our results show that using LLM for SATD category classification is possible. However, to get reliable results, special care should be taken to do the classification over several rounds. Simply instructing LLM to classify comments into different categories does not work well out of the box, as shown by Fleiss' kappa (0.38) after the first round of classification. Utilising another classification round and limiting the choices of ChatGPT to those from the first round from all annotators increases Fleiss' kappa dramatically (0.83), causing it to re-evaluate its earlier answers.

There are cases where ChatGPT has hallucinated when classifying a comment. Specifically, it suffers from faithfulness hallucinations with context inconsistency (Huang et al., 2023). This happens when the comment is incomplete or does not follow a typical writing pattern. The examples mentioned in Section 3 demonstrate this behaviour. The comment "`TODO: make these parameters:`" indicates a flaw in function implementation, indicating Code and Design Debt. However, ChatGPT cannot predict what it should do, as evidenced by its answers. In the first round of annotation, the answer was:*"The comment includes the keyword 'TODO' followed by a task description 'make these parameters'. This comment falls under the Test Debt category because it indicates that the parameters need to be modified or updated for testing purposes. It is a reminder to make changes to the code to properly set up the testing environment"*. Then, when asked to reconsider its answer on the second annotation round, it came up with an answer which did not fit to the suggested categories: *"TODO comments typically indicate areas of the code that require modification or improvement. Since the comment does not provide further details on whether it pertains to code modifications, design improvements, or testing enhancements, it can be classified as Unknown Debt"*. This shows, that there exist cases where ChatGPT does not comprehend the meaning of the comment. The second human annotator agreed with the human annotators interpretation of this comment.

The second case, where the comment does not follow a typical written sentence format, also causes similar issues. An example comment such as "`/* XXX range-check value? */`" is labelled as Defect and Design Debt by the human annotator, as it indicates a possible bug or defect in that function which can require bigger changes in design to fix. However, in the first annotation round ChatGPT labels it as Unknown Debt, stating: "*The comment /* XXX range-check value? */ contains the keyword XXX, indicating that there may be an issue in the code. However, without further context or information, it is unclear what exactly the issue is or which category it falls into. Therefore, this comment falls under the category of Unknown Debt*". In the second round, it still holds on to its original classification, this time answering that "*Defect Debt: The comment suggests that there may be a potential issue with the range-check of a value. If this is indeed a bug or a failure in the software system, it would fall under the category of Defect Debt. - Design Debt: The comment questions whether the range-check of a value is correctly implemented, indicating a potential violation of good design principles. If the design of the range-check is found to be a technical shortcut or a violation of good design practices, it would be classified as Design Debt. - Unknown Debt: The comment contains the keyword XXX without any further explanation. Since it is not clear whether it refers to a defect or design issue, it can be categorised as Unknown Debt. Classification: Unknown Debt*" Again, the second human annotator agreed with the human annotator with the given classification.

4.3. SATD categories in drone systems (RQ3)

Our research discovered that the most prevalent SATD categories in the drone repositories were Code Debt, Unclassifiable Debt, and Design Debt. We again compare our results with previous research, listed at the beginning of Section 4. The results are presented in Table 14.

An exploratory study analysed 37,342 Android repositories written in Java (Wilder et al., 2023). Instead of labelling all the comments, the authors took a random sample of 386 comments labelled as SATD and labelled them into one of 7 categories. This sample size guarantees a 95% confidence level with 5% of confidence. They include "Unclassifiable" as one category, meaning comments that are too vague, incomplete, or not descriptive enough to be assigned to a definitive class. The largest category in their work is Code Debt, followed by Design Debt and Requirement Debt.

Research of SATD in deep-learning frameworks analysed 7 different repositories, from which four were written in combination of Python and C++, one in Python, one in Java and one in C++ (Liu et al., 2020). They labelled the comments belonging into one of the five categories presented in Maldonado and Shihab (2015): Design Debt, Defect Debt, Requirement Debt, Test Debt and Defect Debt. In addition, the authors propose two new classification categories corresponding directly to deep-learning: Algorithm Debt and Compatibility Debt. Their largest category was Design Debt, followed by Requirement Debt ad Algorithm Debt. It is worth noting that the authors do not provide direct numbers for each category, but rather report percentages of each debt present per project. It is possible to construct the numbers for each debt using the information from their paper. However, due to the rounding used in reporting percentages, our final numbers are off by 1 SATD comment in total (7158 SATD comments).

Previous research into SATD in Java projects analysed 10 projects belonging to different application domains (Maldonado et al., 2017). For their work, we calculate the number and percentages for different SATD categories directly from the dataset published by the authors, as the numbers are not available in the published paper. The authors note that they could only identify 5 types of the classifications presented in previous research (Alves et al., 2014): Defect Debt, Design Debt, Documentation Debt, Requirement Debt, and Test Debt. Their dataset has a total of 4071 SATD comments. The largest category is Design Debt, followed by Requirement Debt and Design Debt.

A work into Ethereum contracts looks into SATD in 726,235 contracts written in Solidity (Ebrahimi et al., 2023). They have 229 labelled instances that do not include duplicates. Their largest category is Code Debt, followed by Requirement Debt. These two count for over 80% of all the SATD comments in their dataset.

One previous research investigates SATD in Dockerfiles (Azuma et al., 2022). They analysed 1250 Docker images and discovered a total of 50 SATD comments. The majority of the comments were in the Code Debt category. The second largest category is Test Debt; finally, Defect Debt comes in third. Notably, they also include "Unclassifiable" as one category, where they group comments which are too vague to be classified into any category.

Table 14
SATD category summaries in research.

Domain	Drones[a]	Android apps[b]	Java Projects[c]	Deep-Learning Frameworks[4]	Ethereum Contracts[d]	Docker Files[e]
Language	C, C++	Java	Java	Python, C++, Java	Solidity	Script
Architectural Debt	122 (2.6%)	–	–	–	–	–
Build Debt	75 (1.6%)	–	–	–	–	–
Code Debt	1643 (35%)	136 (35.2%)	–	–	111 (48.5%)	26 (52%)
Defect Debt	558 (11.9%)	40 (10.4%)	472 (11.6%)	428 (6%)	–	6 (12%)
Design Debt	703 (15%)	91 (23.6%)	2703 (66.4%)	4203 (58.7%)	16 (7%)	2 (4%)
Documentation Debt	58 (1.2%)	11 (2.8%)	54 (1.3%)	102 (2.5%)	4 (1.7%)	–
Requirement Debt	652 (13.9%)	77 (19.9%)	757 (18.6%)	1034 (14.4%)	76 (33.2%)	–
Test Debt	120 (2.6%)	5 (%)	85 (0.1%)	419 (5.9%)	19 (8.3%)	9 (18%)
Algorithm Debt	–	–	–	649 (9.1%)	–	–
Compatibility Debt	–	–	–	323 (4.5%)	–	–
Process Debt	–	–	–	–	–	4 (8%)
Unclassifiable Debt	766 (16.3%)	26 (6.7%)	–	–	–	3 (6%)
Forward compatibility Debt	–	–	–	–	3 (1.3%)	–
Total	4697	386	4071	7158	229	50

[a] Our Dataset.
[b] Wilder et al. (2023).
[c] Maldonado et al. (2017).
[d] Ebrahimi et al. (2023).
[e] Azuma et al. (2022).

Table 15
Co-occurring SATD categories.

	Arc.	Build	Code	Defect	Design	Doc.	Req.	Test	Unc.
Architectural	–	1	2	3	0	0	8	0	0
Build	–	–	2	1	0	0	1	2	0
Code	–	–	–	18	108	0	9	6	8
Defect	–	–	–	–	11	2	17	3	2
Design	–	–	–	–	–	0	88	0	1
Documentation	–	–	–	–	–	–	1	0	0
Requirement	–	–	–	–	–	–	–	1	0

4.3.1. Co-occurring SATD categories

In the dataset, there were several cases when a KL-SATD comment was labelled into two distinct categories. This was done as it was seen to portray characteristics of both categories. As per previous research in OBrien et al. (2022), we assigned such comments labels from both categories. However, it can be beneficial to look at which categories tend to co-occur together? Table 15 lists all the co-occurring categories and the times they occurred together.

As we can see, the most co-occurring category is Code Debt and Design Debt. This is explained by the ambiguity of the comment, and whether the comment warrants a small code change or a larger one. Second most common occurrence comes from Design Debt and Requirement Debt. In these cases, the ambiguity comes from interpreting the comment either as a missing feature or something that has been already done and needs to be changed. The rest of the comments follow a similar pattern, where the comments have features of both categories. All of the disagreement cases can be found in the replication package.

4.4. Drone-specific SATD (RQ4)

Our research places drone-specific SATD around 21%. When considering Docker files, there were 36% comments out of 50 SATD comments deemed specifically Docker-specific (Azuma et al., 2022). From deep learning applications, only one classification, algorithm debt (13% of all debts), can be said to relate specifically to that domain (Liu et al., 2020). Domain specific SATD percentages seem to vary between 13%–36% and call for more research to be done in various domains.

4.5. Application and SDK difference (RQ5)

Our dataset included both application and SDK repositories, and the results show that none of the categories showed statistical differences after sequential Bonferroni correction. However, we do not rule out that there would now be differences. Section 4.8.5 discusses the sequential Bonferroni correction and the possible bias it can include into the results. Therefore, more investigations are needed to see whether SDKs truly differ from other applications.

Our work shares similarities in this regard with previous work investigating SATD in machine learning repositories (OBrien et al., 2022). In this work, the authors did a similar split in their dataset. They divided repositories into either machine learning applications or machine learning tools reported differences between two categories in them. However, they do not report statistical numbers, so this difference is not necessarily real.

4.6. Implications

Implications to practitioners. Our answer to RQ3 shows many KL-SATD comments labelled as Unclassifiable Debt. The result means that the contents of the comments are too vague to offer any meaningful insight to anyone other than the writer. This vagueness can pose severe problems in the future, especially if the person responsible for the comment is not available to explain the meaning behind it. This behaviour calls for developing either better comment-writing practices or alternative workflows where the problems are documented on code and in other systems with clear explanations.

Implications to researchers. Our sample study on SATD in drone systems fills a gap in scientific literature by being the first to look into SATD in drone systems. As most TD research is done in projects written in Java, our project presents an important addition to scientific knowledge by presenting analysis from drone projects written mainly in C and C++. Specifically, our work can be a starting point for building more extensive and comprehensive datasets for C and C++ projects.

Our answer to RQ3 shows a need to rethink how KL-SATD and SATD classification is done in research. The previous works (Alves et al., 2014; Li et al., 2015) have laid the foundation for classification. However, recent research has shown a need for new classifications. Our research along with (Azuma et al., 2022; Wilder et al., 2023) have used the "Unclassifiable" category, where a comment is too vague in its

contents to be labelled into any category. This finding has now been shown to occur repeatedly, and it warrants research into the reasons behind such vague comments.

Finally, we present a workflow incorporating ChatGPT or other LLM models as part of the research process. This follows the concept of AI-augmented development lifecycle (Ozkaya, 2023) and takes it to the research context, partially eliminating one of the most expensive stages of research, specifically the manual annotation process.

However, our research shows that this incorporation requires careful planning in order to succeed. The key insight when working with ChatGPT was that annotations must be done in two rounds. In addition, when creating the prompts, it is important to make the model go through a mandatory inner dialogue with itself. Forcing it so explain the choices it would consider and then making it choose from these choices greatly reduces its tendency to jump on the first suggestion it prints out. This suggestion works as a universal guideline when incorporating LLMs to annotation tasks.

4.7. Related work

Here, we discuss two main topics related to ours. First, we discuss the generic categorisation of SATD comments, domain-specific SATD comments, and the application of ChatGPT in scientific research.

Generic SATD comment categorisation. The categorisation of SATD comment is based on two seminal works, which independently identified several shared categories of SATD (Alves et al., 2014; Li et al., 2015). The categories shared by both papers are Architectural Debt, Build Debt, Code Debt, Defect Debt, Design Debt, Documentation Debt, Requirements Debt and Test Debt. There is disagreement in the research community about whether defects should be considered as TD or not. However, a recent software industry survey found that developers themselves do not make a clear distinction between TD and defects (Codabux et al., 2017). Therefore, we also consider defects as TD per the original classifications in Alves et al. (2014), Avgeriou et al. (2016). Finally, previous works (Wilder et al., 2023; Azuma et al., 2022) have discovered the problem of vagueness or incompleteness in some code comments, which can make them impossible to categorise. Therefore, they have utilised a new category, "Unclassifiable Debt", for such comments. We also included this category in our classification scheme.

Domain-specific SATD comment categorisation. In addition to generic SATD, previous works have investigated SATD in specific domains such as Docker files (Azuma et al., 2022) and deep learning applications (Liu et al., 2020). The research done on Docker files also discovered that not all SATD instances were specifically related to Docker files. Instead, a portion of Code Debt, Test Debt, Design Debt and Process Debt comments were identified as Docker specific (Azuma et al., 2022). This is similar to our research, where we show that 22% of SATD comments are related specifically to drones.

The research on deep learning applications does not specify any subtypes of SATD, but proposes that algorithm debt and compatibility debt should be added to SATD categories for this domain. However, their original uses a much more restricted set of SATD categories to begin with than our work. While our work does not go so deep in Drone-specific SATD comments, our work uses the higher level classification (e.g., Code Debt, Defect Debt) used in previous research. In effect, our work lays the foundation for a similar investigations in the drone domain.

SATD studies in C/C++/C#. Although there exists a plethora of SATD studies, most of them are focusing on projects written in Java. As our work deals with drones, the majority of the projects use either C, C++ or C# as their language. There has been relatively few studies done on SATD in these languages. One previous research looks into deep-learning frameworks which were partially written in C++ (Liu et al., 2020), another one focuses on embedded projects written in C in an industry setting (Li et al., 2022), while another recent research investigates the impact of compiler warnings to code quality in projects written in C++ (Johansson et al., 2024).

Only the research looking into deep-learning is partially comparable to our work. However, only one of the projects in their research is written purely in C++ and how significant role of C++ has in other projects is not reported. Neither of the other two works are directly comparable to our work. The research done in an industry setting utilises a deep learning model trained to detect one of four types of SATD with a moderate F1 score (0.666) from code comments (Li et al., 2022). The focus of the research is not in accurately identifying SATD types and amounts, but rather they focus on practitioners and their perception of SATD. The research examining the effect of compiler warnings to code quality utilises open source projects written in C++, and they analyse the projects with SonarCloud (Johansson et al., 2024). However, again the focus is not on investigating SATD types and amounts, but rather looking at how the overall number of different TD metrics (number of code smells, vulnerabilities/etc.) in a project relate to different warning flags.

Application of ChatGPT. Since its release, ChatGPT has been utilised in different research fields, such as medicine and software engineering. In medicine, the usability of ChatGPT as a support tool when making decisions relating to breast tumours was examined (Sorin et al., 2023). The results show that the clinical recommendations given by ChatGPT were in line with those of the tumour board in 70% of cases.

Previous work in software engineering has researched ChatGPT from several different points of view. One research looked into using ChatGPT as an assistant tool for programming (Tian et al., 2023). The results show ChatGPT can generate code for different coding tasks of various difficulty, repair common programming assignments, and reason the original intentions behind code. However, ChatGPT is having trouble generalising to new problems it has not yet seen, and giving just generic problem descriptions hurts ChatGPT's performance.

Earlier research proposed prompt patterns for tasks related to software engineering, such as creating APIs, refactoring code, and providing architectural recommendations (White et al., 2023). Following their idea for different prompt patterns, we created prompts for our different tasks when working with ChatGPT. We also followed the advice from previous research to keep the prompts concise to improve ChatGPT performance (Tian et al., 2023).

4.8. Threats to validity

Here, we discuss possible threats to validity, starting with aspects relating to repository and LLM selection, limitations on focusing only on KL-SATD comments, SATD classification, statistical methods, and ending with questions about reliability.

4.8.1. Repository selection

Our repository selection can be seen as a threat to validity. We did not perform pre-defined pruning of the repositories regarding age or size. Our approach looks into the most popular drone repositories in Github during the time of the writing. This follows the reasoning proposed in earlier research (Ray et al., 2014), where stars are used to measure a given project's popularity. Although the repository selection was uncomplicated, it includes repositories used in previous research like Ardupilot and PX4-Autopilot used in Wang et al. (2021) and Di Sorbo et al. (2023). Like in previous research (OBrien et al., 2022), we manually included tool repositories in addition to open-source applications. In our case, these are the SDK repositories developed by DJI. We consider the number of repositories sufficient for an exploratory study.

Several of the downloaded repositories included several programming languages, some of which were outside the capabilities of srcML.

However, we still considered them viable if the majority of their files were written in the four languages srcML can parse.

Finally, our dataset includes 18 repositories in total. This number is in line with previous works investigating technical debt (e.g., Tsoukalas et al., 2020; Maldonado et al., 2017).

4.8.2. LLM model selection

Our work is based on the use of ChatGPT 3.5. It is possible that higher Fleiss' kappa numbers could have been received while using later version of ChatGPT. However, our purpose was to evaluate whether or not LLM models could be used in classifying KL-SATD comments. The results of Fleiss' kappa show that ChatGPT version 3.5. can be used for that purpose using an annotation procedure consisting of initial annotation round and a later discussion round.

Since we have now set the baseline with ChatGPT 3.5., any later ChatGPT model can be used for the same purpose. The annotation process does not change for later versions, as the disagreement discussion round is always needed even when dealing with human annotators.

4.8.3. Limitations of KL-SATD comments

Our work focuses purely on KL-SATD comments, which can be considered a limitation. By definition, our decision means that we are missing the SATD comments which do not include any of the mentioned keywords. This can have an effect on the overall SATD category amounts. How much comments we are missing we cannot say for sure. Previous research (Rantala et al., 2020) shows that up to 30% of comments can be missing SATD keyword. Of course, this amount can vary between projects. Nevertheless, we consider our work valid in regards to analyses pertaining specifically to KL-SATD comments.

4.8.4. Classification

The classification of our work can be seen as a limitation. We relied on the existing classification scheme. It is possible that there could have existed a previously unknown classification. The two original research papers on classification have created reasonably extensive lists of SATD categories. However, we added "Unclassified Debt" to the scheme as an additional category since it has also been shown to occur in previous research.

It is possible that some of the comments we have categorised as "Unclassified Debt" could be fitted into some other classes. We elected to be on the safe side, and if there was any doubt about the correct class, we categorised the comment as "Unclassified Debt".

In order to guarantee classification reliability, an second human annotator annotated the disagreement cases between original human annotator and ChatGPT. The results show a moderate agreement with the human annotators initial classification, and a poor agreement with ChatGPT.

Finally, another threat to our validity relating to classification is the interpretation of Fleiss' Kappa numbers on rater agreement. However, these benchmark numbers are widely used in software engineering, see e.g. Biswas et al. (2022).

4.8.5. Statistical methods

To ensure more robust results for RQ1, RQ4 and RQ5, we employed sequential Bonferroni correction in each case. For RQ1, it was used to evaluate whether there exists actual differences betewen projects, and for RQ4 and RQ5, sequential Bonferroni correction was calculated for each SATD category. Sequential Bonferroni correction is a very conservative method, and it can reject possible differences too harshly (Cabin and Mitchell, 2000). Due to this conservative nature, the method can inflict bias in the results. To alleviate this, we also report all the p-values without this correction, as this is deemed good and more informative scientific practice (Nakagawa, 2004).

4.8.6. Reliability

Reliability ensures that other researchers can replicate our study's results. We have described the process and uploaded all the necessary code, files and instructions to the replication package. The package also includes files containing the final results of our annotations, allowing easy reviewing without having to go through all the steps of the process. Therefore, with all this information, researchers can replicate our steps.

However, due to the probabilistic nature of ChatGPT, we cannot guarantee that future replication studies of our work will yield precisely the same results. There is no possibility of locking the ChatGPT so that it would give the same results every time.

5. Conclusion

In this work, we performed a sample study on SATD in drone-related open-source repositories. In addition, we evaluated LLM's (ChatGPT 3.5) suitability as a classification tool for SATD. By answering our five RQs, we have uncovered eight different findings.

The first finding is that KL-SATD comments amount to approximately 1% of all the comments present in the repositories which fits the generic range of SATD comments from earlier research. In addition, the KL-SATD comments are localised in few files with the projects and generally these files have only one or two KL-SATD comments in them.

Second finding shows that projects differ from each other when looking at KL-SATD distributions in them. However, we were able to show statistical difference only between two projects.

Third finding shows that utilising LOC rather than number of non-SATD comments is a better predictor for overall number of KL-SATD comment amounts. The high variance-inflation factor indicating multicollinearity between LOC and number of non-SATD comments deters from using both of them as predictors.

Fourth finding shows that utilising LLM (ChatGPT 3.5) improves classification accuracy, as both the human and the LLM can adjust their initial classifications in the second round. This adjustment occurred in 1% of cases for humans and 36% for the LLM. Our work lays a foundation for understanding ChatGPT version 3.5 in this context. Importantly, our classification process is also valid for later versions. As models grow larger, we expect that LLMs will need to adjust their ratings less frequently in the second classification round.

Fifth finding shows that the agreement between human annotator and ChatGPT 3.5 varies depending on the SATD category being labelled. ChatGPT 3.5 is prone to misclassify SATD as "Unclassifiable", when classifying Code Debt comments.

Sixth finding shows that the largest SATD categories in our dataset are Code Debt with 35% of SATD classifications, Unclassifiable Debt (16%), and Defect Debt (15%). In total, these three categories constitute 66% of all SATD classifications.

The seventh finding shows that from our random sample of 385 comments, 82 (21.3%) are related to drones. Statistically, the comments related to drones had higher amount of Requirement Debt comments in them than other comments.

Finally, the eight finding shows a possible difference between drone applications and SDKs when comparing KL-SATD amounts per SATD category. However, the results do not hold after utilising sequential Bonferroni correction.

Overall, the results give a broad overview of KL-SATD comments in drone systems. They also point towards future research possibilities. The first one is to create a more extensive dataset by adding more drone-related repositories into it. Secondly, we want to broaden the SATD comment analysis and to include comments not labelled with a KL-SATD keyword. Third option is to deepen the analysis of SATD comments by performing content analysis on them. This can help to find out what sort of problems comments directly related to drones depict.

Finally, our work shows statistical difference in Requirement Debt amount related directly to drones. This finding requires further qualitative analysis into the comments themselves. This can have implications on drone functions, and therefore in itself warrants a more detailed research into them. Overall, whether there exists a qualitative difference in the contents between drone-related comments and generic comments is a future research idea worth investigating.

CRediT authorship contribution statement

Leevi Rantala: Writing – original draft, Investigation, Software, Data curation, Validation, Conceptualization, Writing – review & editing, Visualization, Methodology, Formal analysis. **Lwin Khin Shar:** Methodology, Writing – original draft, Conceptualization, Writing – review & editing, Supervision, Project administration. **Mika V. Mäntylä:** Writing – review & editing, Supervision, Conceptualization, Writing – original draft, Project administration, Methodology. **Wei Minn:** Validation, Investigation. **Yan Naing Tun:** Investigation, Validation.

Funding sources

Funding: The authors acknowledge the financial support by the Research Council of Finland [grant ID 328058] and Kaute [Tutkijat Maailmalle, grant 20220020].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code to produce the results and the final results are available through Figshare in address: https://figshare.com/s/1870bcc7e6c1cefff63c.

References

Alhefdhi, A., Dam, H.K., Ghose, A., 2024. Towards automating self-admitted technical debt repayment. Inf. Softw. Technol. 167, 107376.
Alves, N.S., Ribeiro, L.F., Caires, V., Mendes, T.S., Spínola, R.O., 2014. Towards an ontology of terms on technical debt. In: 2014 Sixth International Workshop on Managing Technical Debt. IEEE, pp. 1–7.
Avgeriou, P., Kruchten, P., Ozkaya, I., Seaman, C., 2016. Managing technical debt in software engineering (dagstuhl seminar 16162). In: Dagstuhl Reports, Schloss Dagstuhl-Leibniz-Zentrum Fuer Informatik.
Azuma, H., Matsumoto, S., Kamei, Y., Kusumoto, S., 2022. An empirical study on self-admitted technical debt in dockerfiles. Empir. Softw. Eng. 27 (49).
Bavota, G., Russo, B., 2016. A large-scale empirical study on self-admitted technical debt. In: Proceedings of the 13th International Conference on Mining Software Repositories. pp. 315–326.
Bird, C., Ford, D., Zimmermann, T., Forsgren, N., Kalliamvakou, E., Lowdermilk, T., Gazit, I., 2022. Taking flight with copilot: Early insights and opportunities of ai-powered pair-programming tools. Queue 20, 35–57.
Biswas, S., Wardat, M., Rajan, H., 2022. The art and practice of data science pipelines: A comprehensive study of data science pipelines in theory, in-the-small, and in-the-large. In: Proceedings of the 44th International Conference on Software Engineering. pp. 2091–2103.
Cabin, R.J., Mitchell, R.J., 2000. To bonferroni or not to bonferroni: when and how are the questions. Bull. Ecol. Soc. Am. 81, 246–248.
Carletta, J., 1996. Assessing agreement on classification tasks: The kappa statistic. Comput. Linguist. 22, 249–254.
Codabux, Z., Williams, B.J., Bradshaw, G.L., Cantor, M., 2017. An empirical assessment of technical debt practices in industry. J. Softw.: Evol. Process. 29, e1894.
Collard, M.L., Decker, M.J., Maletic, J.I., 2013. Srcml: An infrastructure for the exploration, analysis, and manipulation of source code: A tool demonstration. In: 2013 IEEE International Conference on Software Maintenance. IEEE, pp. 516–519.
de Freitas Farias, M.A., de Mendonç Neto, A.B., Spínola, R.O., 2015. A contextualized vocabulary model for identifying technical debt on code comments. In: 2015 IEEE 7th International Workshop on Managing Technical Debt. MTD, IEEE, pp. 25–32.
DeVore, M.R., 2023. No end of a lesson: observations from the first high-intensity drone war. Déf. Secur. Anal. 39, 263–266.
Di Sorbo, A., Zampetti, F., Visaggio, A., Di Penta, M., Panichella, S., 2023. Automated identification and qualitative characterization of safety concerns reported in uav software platforms. ACM Trans. Softw. Eng. Methodol. 32, 1–37.
Ebrahimi, A.M., Oliva, G.A., Hassan, A.E., 2023. Self-admitted technical debt in ethereum smart contracts: A large-scale exploratory study. IEEE Trans. Softw. Eng..
Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., et al., 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv preprint arXiv:2311.05232.
Iammarino, M., Zampetti, F., Aversano, L., Di Penta, M., 2021. An empirical study on the co-occurrence between refactoring actions and self-admitted technical debt removal. J. Syst. Softw. 178, 110976.
Johansson, A., Holmberg, C., Neto, F.Gomes.De.Oliveira., Leitner, P., 2024. The impact of compiler warnings on code quality in c++ projects. In: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension. pp. 270–279.
Kim, H., Ozmen, M.O., Bianchi, A., Celik, Z.B., Xu, D., 2021. Pgfuzz: Policy-guided fuzzing for robotic vehicles. In: Network and Distributed Systems Security (NDSS) Symposium.
Kruchten, P., Nord, R.L., Ozkaya, I., 2012. Technical debt: From metaphor to theory and practice. Ieee Softw. 29, 18–21.
Kunertova, D., 2023. The war in ukraine shows the game-changing effect of drones depends on the game. Bull. At. Sci. 79, 95–102.
Landis, J.R., Koch, G.G., 1977. The measurement of observer agreement for categorical data. Biometrics 15, 9–174.
Li, Z., Avgeriou, P., Liang, P., 2015. A systematic mapping study on technical debt and its management. J. Syst. Softw. 101, 193–220.
Li, Y., Soliman, M., Avgeriou, P., Somers, L., 2022. Self-admitted technical debt in the embedded systems industry: An exploratory case study. IEEE Trans. Softw. Eng. 49, 2545–2565.
2024. List of unmanned aerial vehicle-related incidents. In: Wikipedia. (Accessed 10 July 2024).
Liu, J., Huang, Q., Xia, X., Shihab, E., Lo, D., Li, S., 2020. Is using deep learning frameworks free? characterizing technical debt in deep learning frameworks. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Society. pp. 1–10.
Maldonado, E.d. S., Shihab, E., 2015. Detecting and quantifying different types of self-admitted technical debt. In: 2015 IEEE 7Th International Workshop on Managing Technical Debt. MTD, IEEE, pp. 9–15.
Maldonado, E.da.Silva., Shihab, E., Tsantalis, N., 2017. Using natural language processing to automatically detect self-admitted technical debt. IEEE Trans. Softw. Eng. 43, 1044–1062.
Nakagawa, S., 2004. A farewell to bonferroni: the problems of low statistical power and publication bias. Behav. Ecol. 15, 1044–1045.
OBrien, D., Biswas, S., Imtiaz, S., Abdalkareem, R., Shihab, E., Rajan, H., 2022. 23 shades of self-admitted technical debt: An empirical study on machine learning software. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 734–746.
Ozkaya, I., 2023. Application of large language models to software engineering tasks: Opportunities, risks, and implications. IEEE Softw. 40, 4–8.
Potdar, A., Shihab, E., 2014. An exploratory study on self-admitted technical debt. In: 2014 IEEE International Conference on Software Maintenance and Evolution. IEEE, pp. 91–100.
Rantala, L., Mäntylä, M., Lo, D., 2020. Prevalence, contents and automatic detection of kl-satd. In: 2020 46th Euromicro Conference on Software Engineering and Advanced Applications. SEAA, IEEE, pp. 385–388.
Ray, B., Posnett, D., Filkov, V., Devanbu, P., 2014. A large scale study of programming languages and code quality in github. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. pp. 155–165.
Rejeb, A., Abdollahi, A., Rejeb, K., Treiblmaier, H., 2022. Drones in agriculture: A review and bibliometric analysis. Comput. Electron. Agric. 198, 107017.
Rice, W.R., 1989. Analyzing tables of statistical tests. Evolution 43, 223–225.
Rios, N., de Mendonç Neto, R.O., 2018. A tertiary study on technical debt: Types, management strategies, research trends, and base information for practitioners. Inf. Softw. Technol. 102, 117–145.
Sharma, R., Shahbazi, R., Fard, F.H., Codabux, Z., Vidoni, M., 2022. Self-admitted technical debt in r: detection and causes. Autom. Softw. Eng. 29 (53).
Sorin, V., Klang, E., Sklair-Levy, M., Cohen, I., Zippel, D.B., Lahat, N.Balint., Konen, E., Barash, Y., 2023. Large language model (chatgpt) as a support tool for breast tumor board. NPJ Breast Cancer 9 (44).
Statista, 2023. Drones - worldwide. In: Statista. (Accessed 10 July 2024).
Stol, K.J., Fitzgerald, B., 2018. The abc of software engineering research. ACM Trans. Softw. Eng. Methodol. (TOSEM) 27, 1–51.
Storey, M.A., Ryall, J., Bull, R.I., Myers, D., Singer, J., 2008. Todo or to bug: exploring how task annotations play a role in the work practices of software developers. In: Proceedings of the 30th International Conference on Software Engineering. pp. 251–260.
Tian, H., Lu, W., Li, T.O., Tang, X., Cheung, S.C., Klein, J., Bissyandé, T.F., 2023. Is chatgpt the ultimate programming assistant–how far is it?. arXiv preprint arXiv:2304.11938.

Tsoukalas, D., Kehagias, D., Siavvas, M., Chatzigeorgiou, A., 2020. Technical debt forecasting: An empirical study on open-source repositories. J. Syst. Softw. 170, 110777.

Wang, D., Li, S., Xiao, G., Liu, Y., Sui, Y., 2021. An exploratory study of autopilot software bugs in unmanned aerial vehicles. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 20–31.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al., 2022. Emergent abilities of large language models. Trans. Mach. Learn. Res..

White, J., Hays, S., Fu, Q., Spencer-Smith, J., Schmidt, D.C., 2023. Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. arXiv preprint arXiv:2303.07839.

Wilder, G., Miyamoto, R., Watson, S., Kazman, R., Peruma, A., 2023. An exploratory study on the occurrence of self-admitted technical debt in android apps. In: 2023 ACM/IEEE International Conference on Technical Debt. TechDebt, IEEE, pp. 1–10.

Xiao, T., Zeng, Z., Wang, D., Hata, H., McIntosh, S., Matsumoto, K., 2024. Quantifying and characterizing clones of self-admitted technical debt in build systems. Empir. Softw. Eng. 29 (54).

Yu, J., Zhao, K., Liu, J., Liu, X., Xu, Z., Wang, X., 2022. Exploiting gated graph neural network for detecting and explaining self-admitted technical debts. J. Syst. Softw. 187, 111219.

Zampetti, F., Fucci, G., Serebrenik, A., Di Penta, M., 2021. Self-admitted technical debt practices: a comparison between industry and open-source. Empir. Softw. Eng. 26, 1–32.

Zampetti, F., Noiseux, C., Antoniol, G., Khomh, F., Di Penta, M., 2017. Recommending when design technical debt should be self-admitted. In: 2017 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 216–226.

Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al., 2023. A survey of large language models. arXiv preprint arXiv:2303.18223.

Leevi Rantala is a Doctoral Researcher of Software Engineering at the University of Oulu, Finland. He received his M.Sc. degree in Software Engineering in 2018 from the University of Oulu. He is interested in researching software development with different machine learning and natural language processing methods, with the focus being on detecting and analysing technical debt.

Lwin Khin Shar (Member, IEEE) is an Associate Professor with the School of Computing and Information Systems, Singapore Management University (SMU). Prior to SMU, he held postdoctoral research positions in University of Luxembourg and Nanyang Technological University (NTU). He received his Ph.D. degree in software engineering from NTU (2014). He His research interests include software engineering, security & privacy, and machine learning, while specialising in analysis of web & mobile applications and recently cyber–physical systems for detecting security vulnerabilities, privacy issues, malware, and anomalies. In his research, he often collaborates with industry partners spanning from healthcare to Government sectors.

Mika Mäntylä is a professor of Software Engineering at the University of Helsinki and University of Oulu, Finland. He received a D. Sc. degree in 2009 in Software Engineering from Aalto University, Finland. His research interests include end-of-lifecycle software engineering activities like software testing, maintenance, and operations. He is also interested in research methods, meta-science, and behavioural and psychological aspects in software engineering. He is keen to utilise natural language processing, classical machine learning, and deep learning for software engineering problems. He has worked as an assistant professor at Aalto University, Finland, and as a post-doc at Lund University, Sweden. He is an associated editor for IEEE Software magazine and Empirical Software Engineering journal. For more information, https://mmantyla.github.io

Wei Minn is currently a Ph.D. student at Singapore Management University School of Computing and Information Systems (SMU SCIS), Singapore. He received B.Sc. degree in Information Systems from SMU SCIS in 2022. His main areas of research interests are in Security Aspects of Software Engineering.

Yan Naing Tun is a Research Engineer at Singapore Management University, specialising in Android security, IoT security, and AI. He holds a bachelor's degree from the University of Computer Studies, Mandalay, and focuses on empirical studies in software testing and security. His work is advancing the field of software security.